EPSR-test-ex56 voids: chained runs with accumulation

Program(s): voids | chained passes: 2 | verification: ALL IDENTICAL (all 2 passes)

What this case tests

Exercises the accumulation branch of `voids` [1]: each run reads the histograms of the previous runs and forms a running average over `nsumt` configurations, and the random-number sequence used to sample the void-pixel correlation continues from the state saved in `randomno.dat`, as in a real multi-configuration analysis.

Code exercised

Fortran (original, fortran_src/):

- `voids.f`: `run_voids`, `find_voids`, `occupypix`, `markpixel`, `search_voids`, `get_void_list`, `ranmine`
- `auxiliary_routines.f`: `initran1`, `ran1`, `saveran1` (generator state in `randomno.dat`)
- `read_and_save_files.f`: `outgofz`, `save_files_others`; `setup_symbols_generic.f`: `setup_symbols_voids`

Rust (port, rust/epsr-core/src/):

- `tools/voids.rs`: `run`, `find_voids`, `occupypix`, `markpixel`, `search_voids`, `get_void_list`
- `rng.rs`: `Ran1::ran1`; `tools/anal_common.rs`: `initran1_file`, `saveran1_file`, `outgofz`
- `symbols.rs`: `setup_input_file`, `read_input_file`, `save_inp_file`; `output.rs`: `r_int`; `symtab_tools.rs`: `VOIDS`

Physical system

Analysed configuration: `200DMSO400Water.ato`, an unchanged copy of the EPSR-test box of 200 dimethyl sulphoxide (DMSO; atom types S, O, C, H) and 400 water molecules (O1, H1): 3200 atoms, DMSO mole fraction 1/3, 300 K, cubic box of side 33.1213 Å (0.0881 atoms/Å^3), the EPSR model of the deuterated DMSO-water neutron data (ISIS SANDALS runs SLS03863, SLS03871, SLS03850), cf. [2]. Pixels within 2.2 Å of the listed atom types are marked (`include S O`, DMSO sulphur and oxygen only; with an `include` list occupied and unoccupied pixels are swapped), connected regions are labelled and their size histogram and pixel-pixel correlation are computed; random origins come from the Park-Miller generator with Bays-Durham shuffle [3,4]. The reference `fort.14` gives a pixel fraction of 0.35069.

Inputs

`chain.VOIDS.n01` and `randomno.dat` were produced by one Fortran run of the same input with `nsumt 0`; the input `chain.VOIDS.dat` (`nsize 80`, `npix 100`, radius 2.2 Å, `include S O`) was then set to `nsumt 99`. `200DMSO400Water.ato` copied unchanged from EPSR-test.

How the accuracy was verified

`commands.txt` (1 program call, repeated 2 times, each pass reading the files written by the previous one) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in all 2 passes.
`repeat 2`: each pass reads the previous histograms (running average with nsum = 100) and continues the random sequence from `randomno.dat`. Because `voids` writes nsumt into the `npix` entry, nsumt itself stays 99 and npix becomes 100, so every pass starts from the same pixel grid and the chain remains valid (reference `.dat`: `npix 100`, `nsumt 99`). Compared after each pass: `chain.VOIDS.n01`, `chain.VOIDS.dat`, `fort.14`, `randomno.dat`. The configuration is the same in every pass, so the chained passes test the read-back of the accumulated file and the running-average arithmetic.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] A. K. Soper and A. Luzar, "A neutron diffraction study of dimethyl sulphoxide-water mixtures", J. Chem. Phys. 97, 1320-1331 (1992).
[3] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find", Commun. ACM 31, 1192-1201 (1988).
[4] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, Numerical Recipes in Fortran 77, 2nd ed., Cambridge University Press (1992).
[5] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.